

Machine Learning

Mikkel Moth Billing
202307989

26. september 2025

Indhold

Indhold	1
1 Linear Models	2
1.1 Decision Trees	3
Entropy	4
Loss Function	5
1.2 Error	6
Loss Function Error	6
1.3 Linear Regression	7
1.4 Logistical Regression	8
1.5 Multinomial Regression	9
2 Non-Linear Models	11
2.1 Growth Function	11
2.2 Testing	12
2.3 Overfitting	12
Regularization	12
Validation	13
3 Neural Nets	13
3.1 Back Propagation	14

1 Linear Models

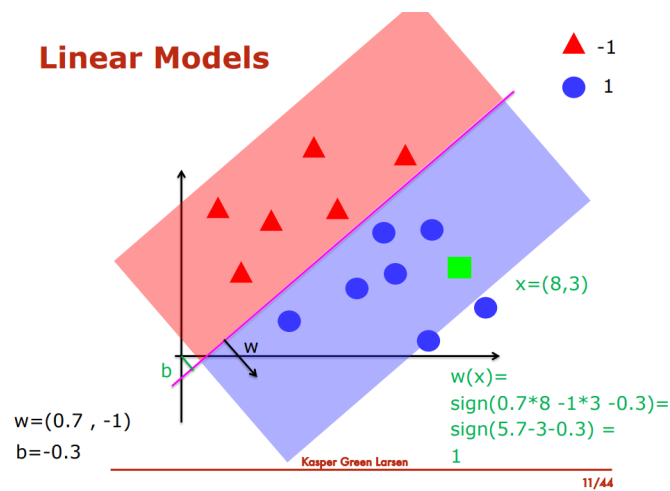
Hypothesis consists of the features vector:

$$x = \{x_1, \dots, x_d\}$$

and a bias b . if the linear combination of the weights and the input, and compare it to the bias, we can make predictions.

Example of mortgage, approve is 1 and decline is -1 . So we can predict that we approve

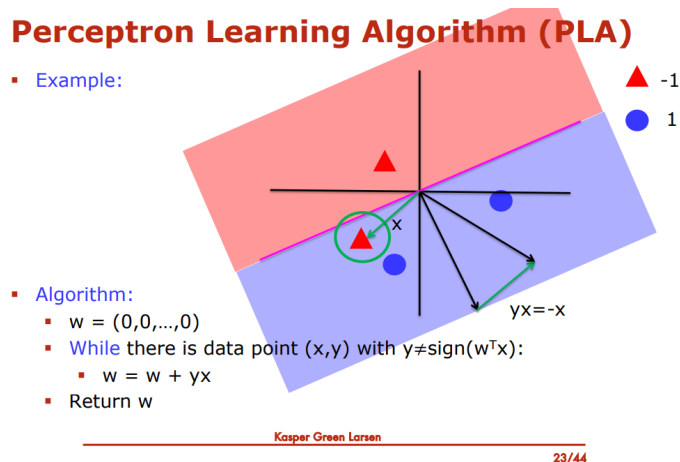
$$w_1x_1 + \dots + w_dx_d > -b$$



To find the parameters we can use **Perceptron Learning Algorithm (PLA)**.

Algorithm: While there is a data point (x, y) with

$$y \neq \text{sign}(w^T x) \rightarrow w = w + yx.$$



As we can see the algorithm can enter an infinite loop, as there might not be a linearly separable dataset.

A way to fix this is to use Pocket PLA which saves the most accurate hyperplane.

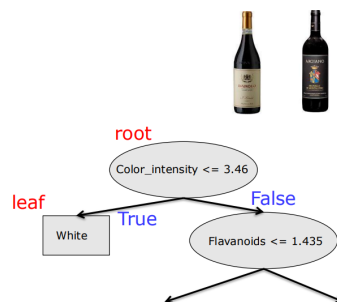
Pocket PLA

- Would like to still do something useful if not linearly separable!
- Pocket version: Remember best w in your pocket.
- Algorithm:
 - $w = (0,0,...,0)$
 - $bestScore = 0$
 - $bestW = w$
 - While you have more time, let (x,y) have $y \neq \text{sign}(w^T x)$:
 - $w = w + yx$
 - $score = \text{percentage of data correctly classified by } w$.
 - If $score > bestScore$:
 - $bestW = w$, $bestScore = score$
 - Return $bestW$

1.1 Decision Trees

Decision Trees

- Classifying wine.
 - Red, White, Rosé
- Predict(x):
 - Start at root node.
 - Test condition in root and descent to relevant child.
 - Upon reach leaf:
 - Return value stored at leaf.
- Node:
 - Question tests **one** feature against **one** value.

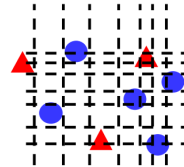
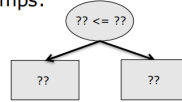


We would like to keep the decision tree small to not overfit to the training data. So we could find the best performing tree with k nodes. But this is **NP-hard**, so we rely on **heuristics**. With a greedy learning algorithm with $k = 2$:

Small Decision Trees

- Greedy Learning Algorithm.

- Decision Stumps:



- Find best decision stump:

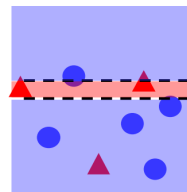
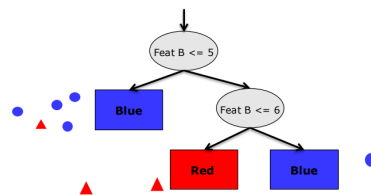
- For each of d features:
 - For each of $n-1$ relevant splits:
 - Try that root, partition points into leaves, assign best leaves, compute accuracy.
- Use best split found.

We test the tree on all possible split, this has $O(n^2d)$ runtime.
 Another way to build a tree is to increase k to maximise the accuracy.

Small Decision Trees

- Final Tree:

$k=2$ example



Accuracy: 7/8

Entropy

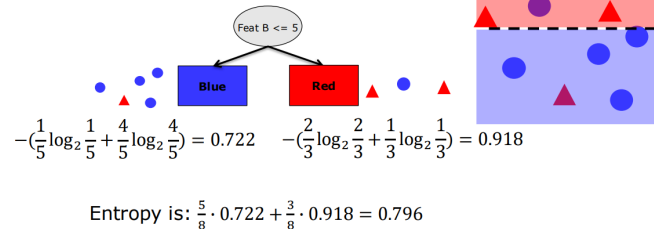
Another way to test decision tree is to base it on entropy. If we have n point with n_i as the numbers of points with label i . And $p_i = n/n_i$. Then entropy is:

$$H(S) = - \sum_{i=1}^k p_i \log_2 p_i.$$

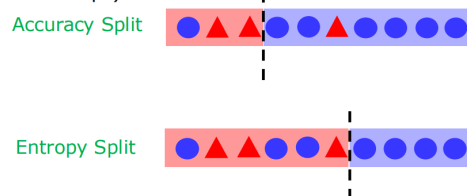
Entropy is a measure of uncertainty in distributions, high entropy is high uncertainty. $H(S)$ describes the expected number of bits it takes to describe the label of a random element drawn from S . If all elements in S are equal $H(S) = 0$. If there is an equal amount $H(S) = \log_2 k$.

Entropy

- Entropy of tree: $\sum_v \frac{|S_v|}{n} H(S_v)$



- May give different trees than using accuracy.
- 1-dimensional example, i.e. one feature.
 - Entropy prefers "pure" splits over accuracy.
 - Seem to work best in practice when constructing decision trees (larger than stumps).



Loss Function

A Loss Function weighs how much a mistake influences a model. We can then try to minimize the loss function over the training points. An example is the Least Square Loss Function:

$$L(y', y) = (y' - y)^2.$$

Small Regression Trees

- Greedy Learning Algorithm.
 - Start in simple case:
 - Our tree may only contain 1 node.
 - Root is a leaf.



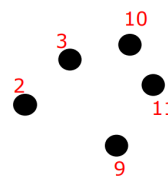
- What is best choice of root?

- The x that minimizes $\frac{1}{n} \sum_{i=1}^n (x - y_i)^2$ is the mean:

$$x = \frac{1}{n} \sum_{i=1}^n y_i.$$

Mean is $(2+3+9+10+11)/5=35/5=7$

Loss is $((2-7)^2 + (3-7)^2 + (9-7)^2 + (10-7)^2 + (11-7)^2)/5=14$.



1.2 Error

In-Sample Error (E_{in}) is the error on the training data, and is what we try to minimize. The Out-Sample Error (E_{out}) is the error on the unknown data, this is what we want to minimize. So we hope that $E_{in} \approx E_{out}$.

Loss Function Error

For the special Loss function 0-1 Loss, we know the error is

$$E_{in}(h) = \frac{1}{n} \sum_{i=1}^n 1_{h(x_i) \neq f(x_i)}.$$

The Out-Sample error is just the probability that $h(x) \neq f(x)$.

The Hoeffding inequality says that the probability of the two errors being further apart than some number ε is

$$P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 2e^{-2\varepsilon^2 n},$$

where n is training examples. If we have multiple hypotheses M we get:

$$P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 2Me^{-2\varepsilon^2 n}.$$

From this we can prove that minimizing $E_{in}(h)$ is a good strategy since for any other h' in H , we get that $E_{out}(h) \leq 2\varepsilon + E_{out}(h')$, so any other h' is at most 2ε better than h .

We can also use Hoeffding inequality to find out what sampling size we should use. If we want the difference in error to be less than 0.1 with 95% confidence, and $M = 1000$, we get:

$$2000e^{-2 \cdot 0.1^2 n} < 0.05 \Rightarrow n > 50 \ln(40.000) = 530,$$
$$n > \frac{\ln(M/(1-c))}{2\varepsilon^2}.$$

If we are using an algorithm with infinite hypothesis, we can use a large number for M , for hyperplanes representable using d floats, that would be $M = 2^{32d}$.

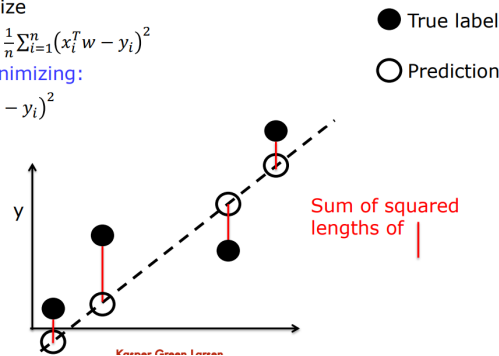
1.3 Linear Regression

The Trick

- **Regression:**
 - Features: $x=(x_1, \dots, x_d)$
 - Labels: R
- **Linear Model:**
 - Parameters of hypothesis w : $(w_1, w_2, \dots, w_d, b)$
 - **Predict:** $w(x) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b = x^T w + b$
- **Trick:**
 - Annoying to have special bias parameter b .
 - Make data $(d+1)$ -dimensional and set first coordinate of all data points equal to 1.
 - Then parameters $(w_0, w_1, w_2, \dots, w_d)$.
 - Predict $w(x) = w_0 \cdot 1 + w_1x_1 + w_2x_2 + \dots + w_dx_d = x^T w$.

Linear Regression

- **Goal:** Minimize
 - $E_{in}(w) = \frac{1}{n} \sum_{i=1}^n (x_i^T w - y_i)^2$
- **Same as minimizing:**
 - $\sum_{i=1}^n (x_i^T w - y_i)^2$



Derivative of Matrix

If $l = (l_1, \dots, l_k)$ is a function from $R^d \rightarrow R_k$, then the Jacobian is a matrix of dimension $k \times d$.

$$\frac{\partial l}{\partial x} = \begin{pmatrix} \frac{\partial}{\partial x_1} l_1(x) & \dots & \frac{\partial}{\partial x_d} l_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} l_k(x) & \dots & \frac{\partial}{\partial x_d} l_k(x) \end{pmatrix}$$

Some helpful tricks are:

$$\frac{\partial}{\partial x} Ax + b = A,$$

$$\frac{\partial}{\partial x} x^T x = 2x^T.$$

Linear Regression

- **Goal:** Minimize
 - $l(w) = (Xw - y)^T (Xw - y) = \sum_{i=1}^n (x_i^T w - y_i)^2$
- **Compute gradient as transpose of matrix of derivatives.**
- **Matrix of derivatives:**
 - $\frac{\partial}{\partial w} (Xw - y)^T (Xw - y) =$
- **Using Chain rule:**
 - $2(Xw - y)^T \frac{\partial}{\partial w} (Xw - y) =$
 - $2(Xw - y)^T X =$
 - $2(w^T X^T X - y^T X).$
- **Gradient as transpose:**
 - $2(w^T X^T X - y^T X)^T = 2(X^T X w - X^T y).$

Rules:

- $\frac{\partial}{\partial x} Ax + b = A$
- $\frac{\partial}{\partial x} x^T x = 2x^T$

Same as we got earlier

$$\Rightarrow w = (X^T X)^{-1} X^T y$$

1.4 Logistical Regression

If we want to calculate a probability based on some data, we use logistical regression. To map a regression to a number between 0 and 1, we use the Sigmoid Function,

$$\theta(z) = \frac{1}{1 + e^{-z}}.$$

So for our prediction function we use:

$$\theta(w^T x) = \frac{1}{1 + e^{-w^T x}}.$$

So what we want is for $\theta(w^T x)$ to be equal to $P[y = 1|x]$, the likelihood of seeing y_i given x_i is $\theta(y_i w^T x_i)$. Then the total likelihood of a data set (S) is

$$P[S|w, X] = \prod_{i=1}^n \frac{1}{1 + e^{-y_i w^T x_i}},$$

which we can then maximise to get our prediction function. We do this by minimizing the average of the negativ log function:

$$\begin{aligned} E_{in}(w) &= -\frac{1}{n} \ln(P[S|w, X]) = \frac{1}{n} \sum_{i=1}^n \ln(1 + e^{-y_i w^T x_i}), \\ \nabla E_{in}(w) &= -\frac{1}{n} \sum_{i=1}^n \frac{1}{1 + e^{-y_i w^T x_i}} e^{-y_i w^T x_i} y_i x_i \\ &= \frac{1}{n} \sum_{i=1}^n -y_i x_i \theta(-y_i w^T x_i). \end{aligned}$$

But we can not set this function to 0 to minimize, so we use Gradient Decent, which is to "roll a ball down a hill", by guessing $w = w - \eta \nabla l(w)$, where η is a step size (usually $\eta = 0.1$) and $\nabla l(w)$ is the gradient of the function. But this can be slow, so we could use Stochastic Gradient Decent, be just taking a batch of datapoints,

$$\begin{aligned} g &= \frac{1}{B} \sum_{i=1}^B -y'_i x'_i \theta(-y'_i w^T x'_i), \\ w &= w - \eta g. \end{aligned}$$

1.5 Multinomial Regression

If we have k different classes that we need to fit for instead of just 1 and -1 . We can use One-in-K Encoding, this means that we use unit vector (instead of just numbers), to classify different targets. For example wine we could have:

Red: (1, 0, 0)
White: (0, 1, 0)
Rose: (0, 0, 1)

So now our label vector turn into a $(n \times K)$ matrix, and our weight vector turn into a $(d \times K)$ matrix. To reduce numbers in a vector to a probability distrubtion we use the softmax function which scales the sum of a vector to 1

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{i=1}^K e^{z_i}}.$$

Which for code implementation could also be calculated as:

$$\text{softmax}(z)_j = e^{z_j - z_a - \ln(\sum_{i=1}^K e^{z_i - e^{z_a}})},$$

where z_a is $\max(z)$

Then for Multinomial Regression our prediction of X $\text{softmax}(XW)$, where we apply the softmax function row-wise. Then the probabilty that we see the correct labels using W is

$$P[S|X, W] = \prod_{i=1}^n y_i^T \text{softmax}(x_i^T W),$$

which we can rewrite to negative log and then minimize:

Multinomial Regression

- K classes, n data points $x_1, \dots, x_n$:
 - $X = \begin{pmatrix} x_{1,1} & \dots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \dots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \dots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \dots & y_{n,K} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \dots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \dots & w_{d,K} \end{pmatrix}$
- Maximize over W
 - $\prod_{i=1}^n y_i^T \text{softmax}(x_i^T W)$
- Negative log likelihood. Scale by 1/n. Min
 - $E_{in}(W) = -\frac{1}{n} \sum_{i=1}^n \ln(y_i^T \text{softmax}(x_i^T W))$
- Luckily, we are told that the gradient is
 - $\nabla E_{in}(W) = -\frac{1}{n} X^T (Y - \text{softmax}(XW))$

Sanity check:
Gradient should have same shape as W

Entry (i,j) equals
 $\partial E_{in}(W) / \partial w_{i,j}$

Then we can do Gradient Decent og Stochastic Gradient Decent

Gradient Descent

- Gradient Descent for Multinomial Regression:
 - Initialize W (e.g. random $N(0,1)$ distributed entries).
 - While more time:
 - $\nabla E_{in}(W) = -\frac{1}{n} X^T (Y - \text{softmax}(XW))$
 - $W = W - \eta \nabla E_{in}(W)$
 - Return W.

Stochastic Gradient Descent

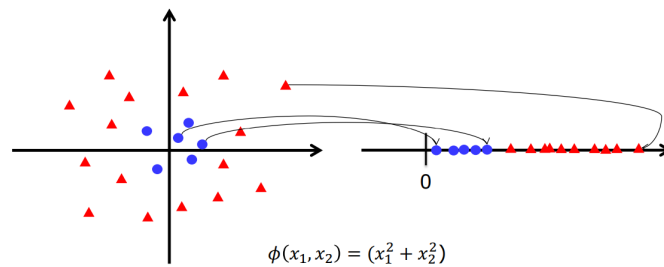
- Mini-batch SGD for Multinomial Regression:
 - Initialize W (e.g. random $N(0,1)$ distributed entries).
 - While more time:
 - Permute $(x_1, y_1), \dots, (x_n, y_n)$ randomly.
 - For $i=1, \dots, n/B$:
 - Let $(x'_1, y'_1), \dots, (x'_B, y'_B)$ be next B data items.
 - $G = -\frac{1}{B} X'^T (Y' - \text{softmax}(X'W))$
 - $W = W - \eta G$
 - Return W.
- Each time we permute is called an **epoch**.
- **Practical**: Reduce η by a small factor after fixed number of epochs.

2 Non-Linear Models

A way to work with data that can not be trained to a hyperplane, like for example a circle, is to transform the feature vector to a new dimension

$$\begin{aligned}\varphi: \mathbb{R}^d &\rightarrow \mathbb{R}^{d'} \\ \varphi(x_1, x_2) &= (x_1^2 + x_2^2).\end{aligned}$$

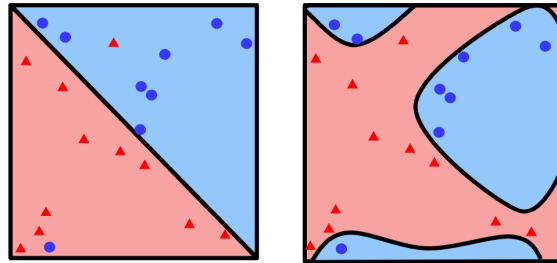
You can then use linear models of the new model.



This can make more accurate models, but if they are too complex they might be more wrong.

Some Considerations

4th degree transform
(1, $x_1, x_2, \dots, x_1^3 x_2, x_1^4, \dots$)



- Not worth high complexity to decrease E_{in} a tiny bit. Probably large E_{out}

2.1 Growth Function

The Growth Function states that there is a finite number of hypothesis sets, dependent on the number of points in the set

$$m_H(n) = \max_{x_1, \dots, x_n \in X} |\{(h(x_1), \dots, h(x_n)) : h \in X\}|.$$

We can relate these hypothesis sets to M in Hoeffdings inequality

$$P(|E_{in}(h) - E_{out}(h)| > \epsilon) \leq 8m_H(2n)e^{-\epsilon^2 n/8}.$$

But it can be hard to find a function for $m_H(n)$, so instead we can bound the growth function with Shattering. If we have k points in a 2D space, we

need to be able to all 2^k dichotomies, meaning that we can create all subsets of points with hypothesis H . (See ML 15 video) If all the dichotomies can be generated the points are shattered. If there is no data set of k points that can be shattered then k is a break point for H .

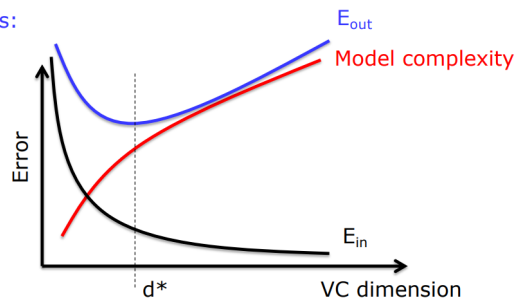
This lead to the Vapnik-Chervonenkis dimension (VC dimension), which is d such that $d = k - 1$. From this we know that $m_H(n) \leq n^d + 1$, then

$$P(|E_{in}(h) - E_{out}(h)| > \varepsilon) \leq 8((2n)^d + 1)e^{-\varepsilon^2 n/8}.$$

Here we can see that given enough data we can always get that E_{in} and E_{out} are close, if the VC dimension is bounded ($d < \infty$).

VC Dimension

- H with VC dimension d has generalization:
 - $\Pr[|E_{in}(h^*) - E_{out}(h^*)| > \varepsilon] \leq 8((2n)^d + 1)e^{-\varepsilon^2 n/8}$
- Conclusions:



2.2 Testing

2.3 Overfitting

Overfitting is when we have a hypothesis that is too complex, this means we get a low in-sample error, but high out-sample error. To solve this we use hyperparameters λ , which is user given parameters that changes something about the learning algorithm. Some examples are:

Regularization

To make sure we don't overfit, we can use regularization. This could be linear regression with weight decay, where we add a limit to the size of the weights. Here we minimize

$$\begin{aligned} & \frac{1}{n} \sum_{i=1}^n (w^T x_i - y_i)^2 + \lambda \|w\|^2, \\ \nabla_w &= \frac{1}{n} (2X^T X w - 2X^T y) + 2\lambda w, \\ \Rightarrow w &= (X^T X + n\lambda I)^{-1} X^T y. \end{aligned}$$

Regularization

- Linear Regression with Weight Decay

- Minimize

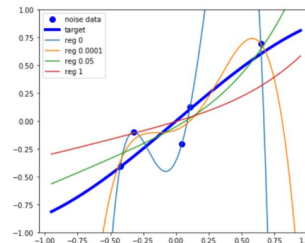
- $$\frac{1}{n} \sum_{i=1}^n (w^T x_i - y_i)^2 + \lambda \|w\|^2$$

- Does regularization help?

- For linear models, it often does.
- Depends a lot on model.

- For linear regression, even has its own name:

Ridge Regression



Validation

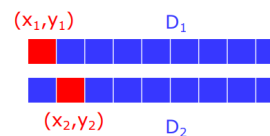
Here we take k inputs from the training data, to use for validation instead. We can then evaluate a $E_{val} \approx E_{out}$, so the learning algorithm can make a decision based on this. We could run the algorithm with different λ , and evaluate all the hypothesis, and then pick the best function. A trick is that after we find λ then we can retain with all the data.

We can also do cross validation to test the hypothesis. Here we train the algorithm on all the data except one point, and then evaluate on that one point. We do this for all points and average the loss over all points. In practice doing it on one point at the time is too slow, so you could split the data in 5, and do it 5 times.

Cross Validation

- Cross validation with $k=1$:

- For each x_i of data items $x_1, \dots, x_n$:
 - Create $D_i = \{x_1, \dots, x_n\} \setminus \{x_i\}$.
 - Train on D_i .
 - Compute validation loss e_i on (x_i, y_i) .
- Set $E_{cval} = (1/n)(e_1 + e_2 + \dots + e_n)$.



- Model Selection:

- Do above for every choice of (hyper-)parameters.

- Observations:

- For each i , use almost all data. Great.
- For each i , validate on only one point. Bad.
- Hope: Averaging gives good estimate.

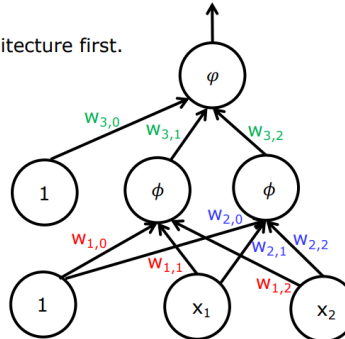
Cannot prove. Not always true.

3 Neural Nets

When building a neural network we first have to look at each neuron. The neuron takes in weights and data, and determines what it should send out to other connected neurons. An activation function φ , it could run could be $\text{ReLU} = \max(0, w^T x)$, or the Sigmoid function.

Neural Net Architectures

- Feed Forward Neural Net.
 - When train, choose architecture first.
 - Nodes
 - Edges
 - Activation Functions
 - Then train weights.



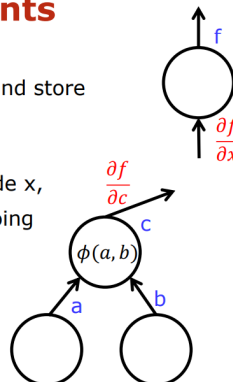
When running a neural network we first decide the architecture, and then the weights gets updated. A simple architecture is fully connected feed forward neural network as seen in the figure above.

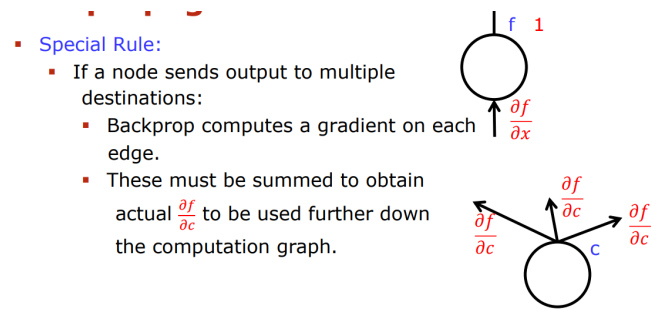
3.1 Back Propagation

If we see the neural net as a function $f(x_1, \dots, x_i)$, where $x_1, \dots, x_i$ all have values, we want to find the numerical value of the partial derivative of the function, with respect to all the parameters. We do this by using the chain rule on all the nodes, and finding their partial derivative.

Computing Gradients

- General Algorithm:
 - Forward pass, compute and store values of all nodes.
- Backprop:
 - Compute $\frac{\partial f}{\partial x}$ for every node x , starting from top and going down.
 - To compute $\frac{\partial f}{\partial a}$:
 - $\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c} \frac{\partial c}{\partial a} = \frac{\partial f}{\partial c} \frac{\partial \phi(a,b)}{\partial a}$
 - Already computed $\frac{\partial f}{\partial c}$
 - $\phi(a,b)$ is simple and a,b known from forward pass.





Because we can find the derivatives we can do stochastic gradient decent.
This can be done with pytorch.